

Bespoke Agentic AI Factory

Composable · Observable · Governed AI pipelines for the enterprise

Agentic RAG **AWS Kiro** **AWS Transform** **Spring AI** **DeepEval Gates**

Solution Architecture · Pipeline Flows · Live Demo

20-slide demo pack

One factory, five governed stages

- Turn scattered enterprise knowledge into grounded, cited answers

5

governed pipeline stages

- Every answer passes automated quality + safety gates before release

- Modernise legacy Java/Spring estates with agentic transforms

2

internal sources wired (Confluence, SharePoint)

- Full observability: metrics, traces and a single pane of glass

- Ships on AWS (ECS Fargate) via Terraform + Harness with canary releases

99

automated tests, all green

4

deploy environments w/ canary

Enterprise AI stalls on trust, silos and legacy

Knowledge silos

- Docs spread across Confluence,
- SharePoint, wikis and drives
- No unified, searchable context

Unverified output

- LLMs hallucinate and drift
- No automated quality gate
- Compliance / audit risk

Legacy estates

- Old Spring Boot, CVE exposure
- Hardcoded rule engines
- Manual, slow modernisation

→ **The gap: a governed, observable pipeline that grounds, verifies and modernises — not just a chatbot.**

Capabilities at a glance



Agentic RAG

Hybrid vector + graph,
multi-hop retrieval



Source connectors

Confluence & SharePoint
→ one RAG index



Eval gates

DeepEval metrics +
LLM-as-judge, CI-gated



AWS Transform

7 modernisation
definitions (atx)



Spring AI

Boot 2.x → 3 +
Spring AI 1.0



Single pane of glass

Metrics, traces,
RED per stage



Secure delivery

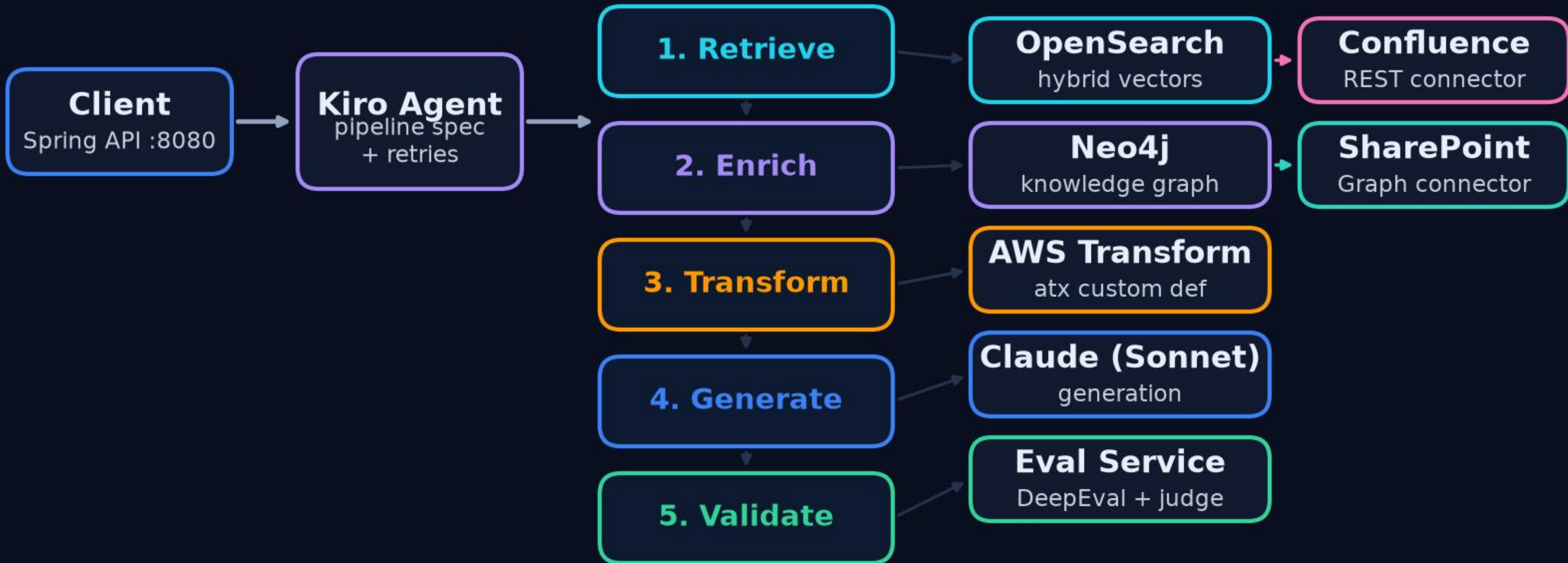
SAST/DAST gates,
canary releases



Knowledge graph

Neo4j entity
enrichment

High-level system architecture



OBSERVABILITY

OTel Collector **Prometheus** **Jaeger** **Grafana** **CloudWatch**

AWS · ECS Fargate · ALB · Terraform · Harness

CORE FLOW

The 5-stage agentic pipeline

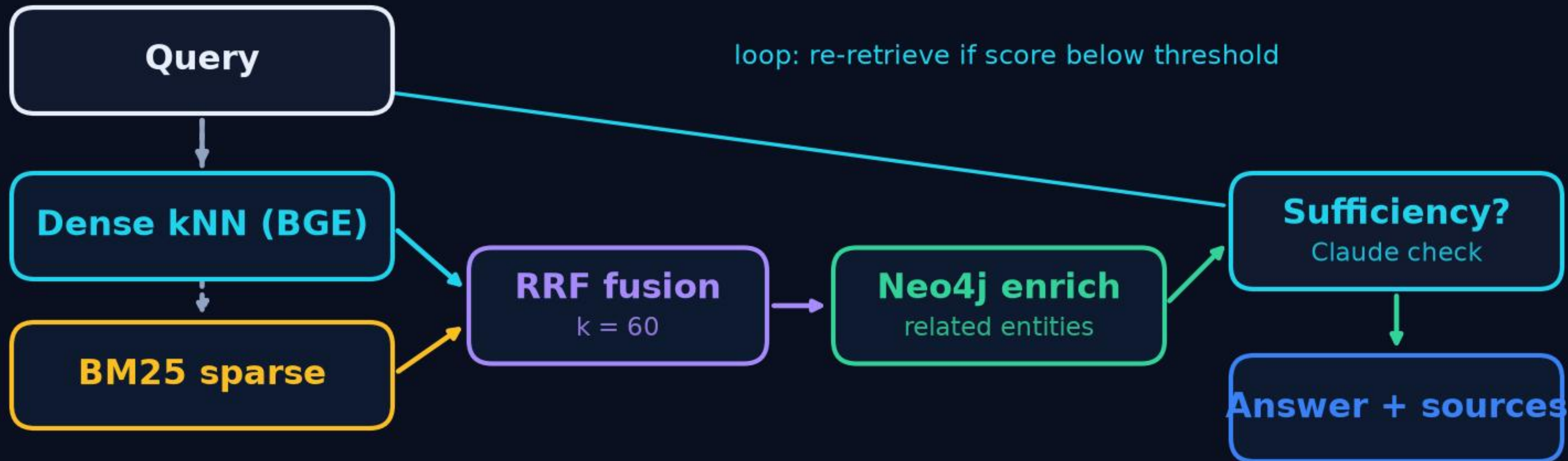
multi-hop: re-retrieve until sufficiency met



Why it matters

- Each stage is an independent, instrumented service (OTel span per stage)
- Retry + sufficiency scoring make retrieval agentic, not single-shot
- The validate stage can block a bad answer before it reaches the user

Agentic RAG — hybrid retrieval + graph



Hybrid

dense + sparse, best of both

Grounded

every claim cites a source

Agentic

multi-hop with sufficiency

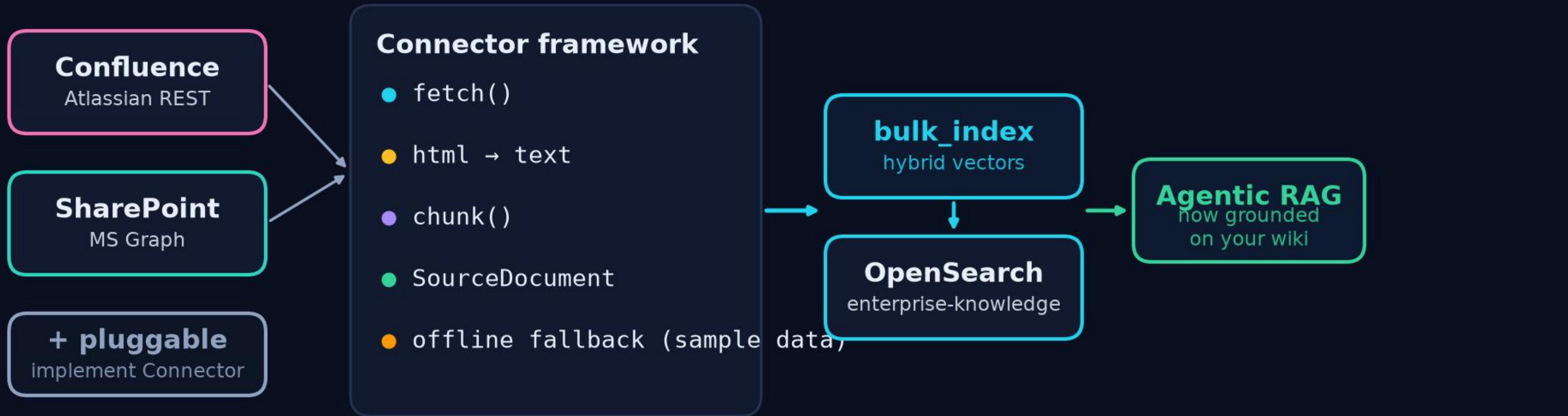
Filtered

metadata + dept scoping

NEW CAPABILITY

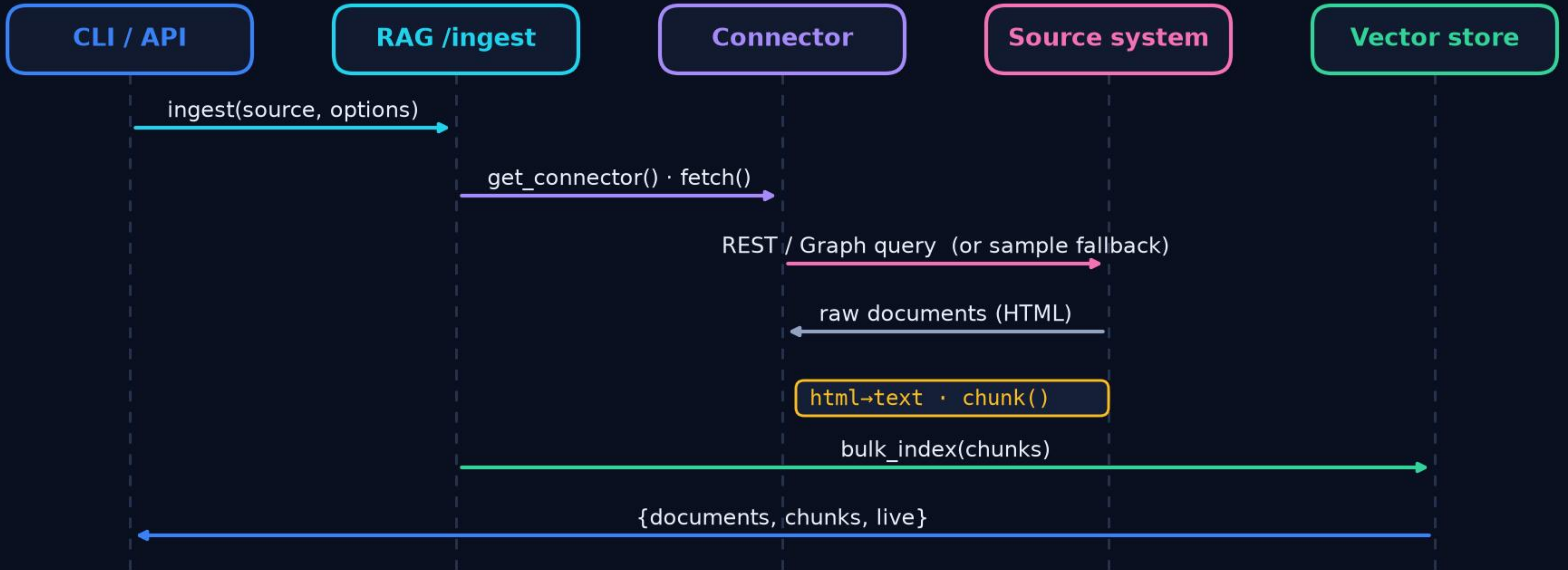
Internal source connectors

Confluence & SharePoint → the agentic RAG index



`POST /ingest {"source":"confluence"}` · `GET /sources` · `python scripts/ingest_sources.py --source sharepoint`

Connector ingestion — sequence



Offline-safe: with no credentials the connector returns bundled sample pages, so the demo always runs.

Evaluation gates — quality before release

Answer relevanc ≥ 0.80
threshold ≥ 0.80

Faithfulness ≥ 0.85
threshold ≥ 0.85

Contextual recal' ≥ 0.70
threshold ≥ 0.70

Hallucination ≤ 0.10
threshold ≤ 0.10

Toxicity ≤ 0.05
threshold ≤ 0.05

Bias ≤ 0.10
threshold ≤ 0.10

LLM-as-judge

- Weighted rubric across metrics
- Self-consistency sampling
- Returns weighted_score + pass/fail
- Runs per answer (validate stage)
- Same suite gates CI deployment

PASS → release · **FAIL** → block + log

AWS Transform Custom — the factory

enterprise-context-normaliser

PII redaction · date norm · truncation

java-spring-to-spring-ai

legacy Spring → Spring AI 1.x

eval-dataset-updater

regenerate eval cases on schema change

fintech-java-spring2-to-spring3

Boot 2.x → 3.2 · WebClient · OTel

fintech-rules-to-agentic-rag

rule engines → Kiro agentic RAG

fintech-soap-to-rest

SOAP → REST anti-corruption layer

fintech-cve-remediation

scan → map CVE → patch, gated to 0 HIGH

Driven by AWS Kiro agent specs · 6-service fintech legacy estate as the modernisation demo

Spring AI modernisation

LEGACY · Spring Boot 2.7

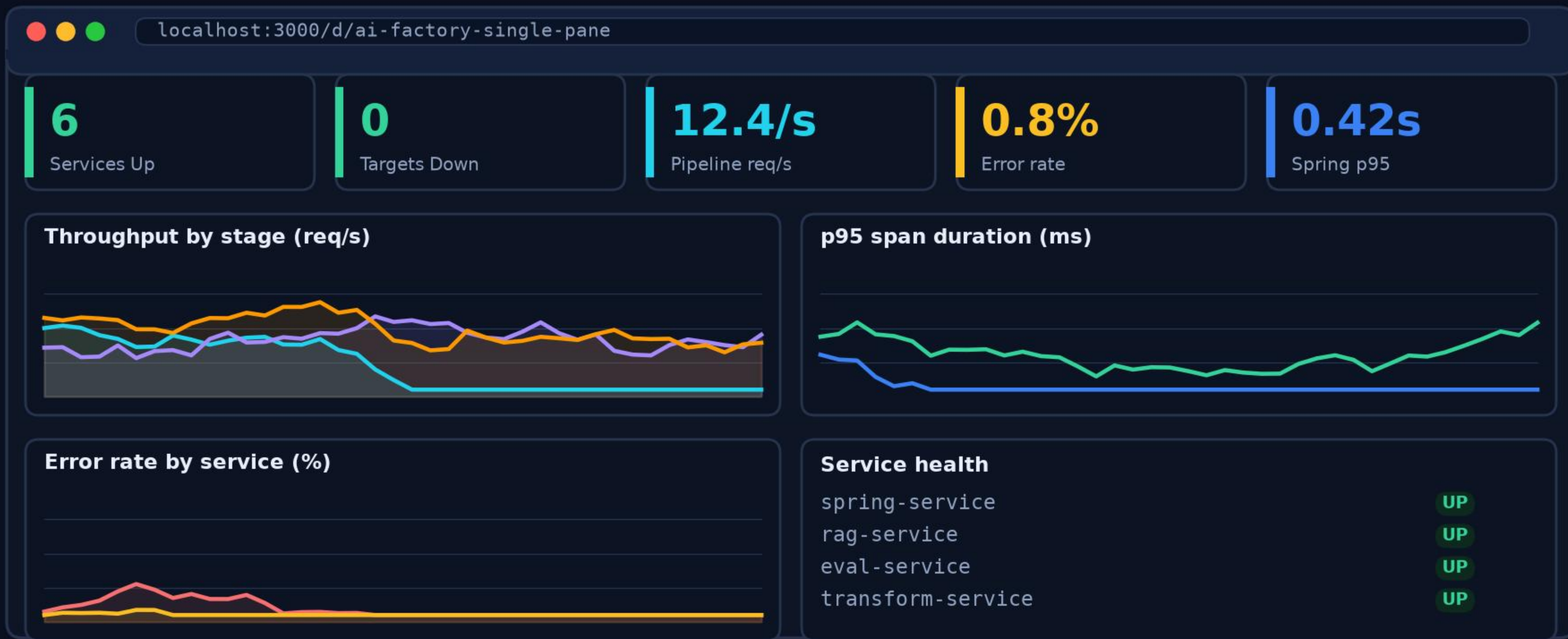
- RestTemplate, blocking I/O
- Solr search client
- Manual prompt strings
- No retries / resilience
- No telemetry, no eval
- CVE-bearing dependencies



MODERN · Spring Boot 3 + Spring AI 1.0

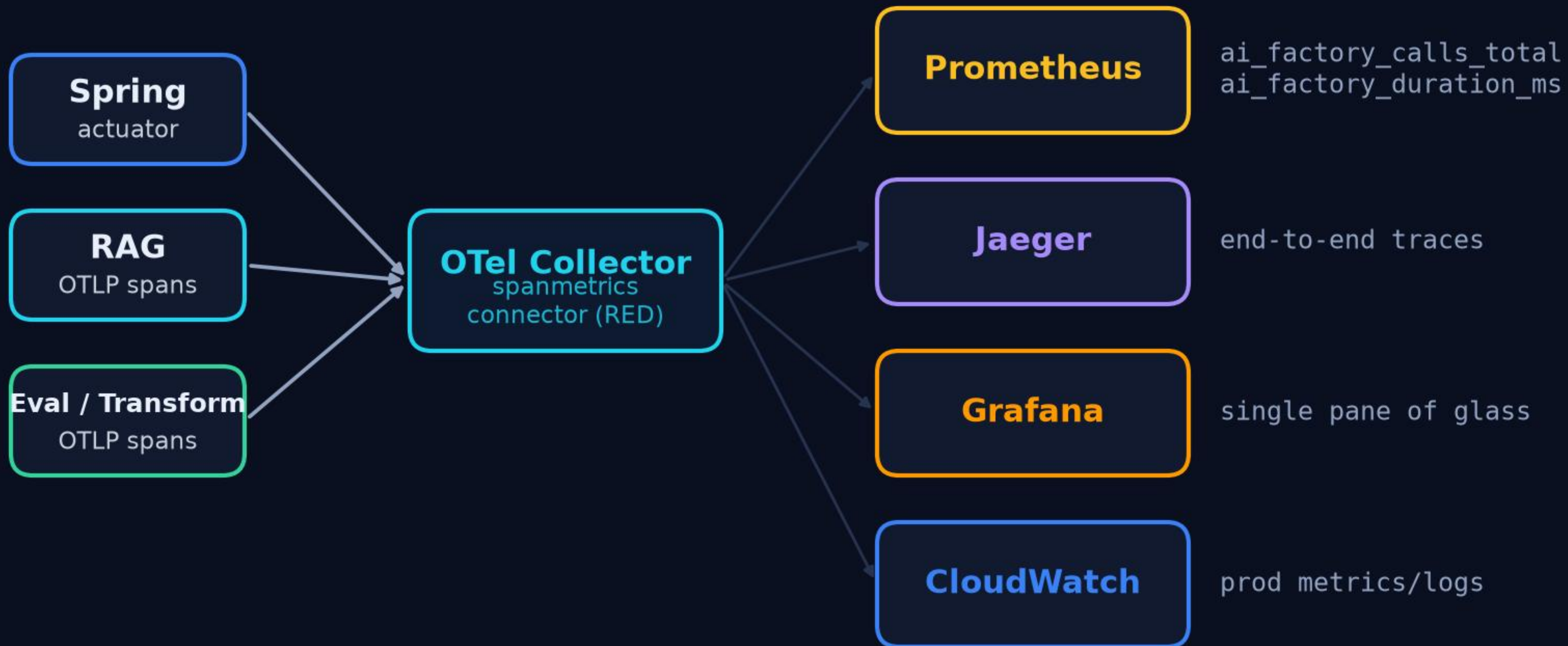
- WebClient, reactive + Resilience4j
- AnthropicChatModel (Spring AI)
- Grounded prompts from RAG
- OTel spans + metrics
- Eval gate before response
- CVE-remediated, 0 HIGH/CRITICAL

Single pane of glass — Grafana



Representative mockup — live dashboard needs `docker compose up` (spanmetrics RED metrics + Spring actuator).

Observability pipeline



Fix shipped: spanmetrics turns the Python services' traces into RED metrics so every stage shows up in Grafana.

AWS deployment architecture

VPC · per environment (dev / sit / preprod / prod)



- One Terraform stack, four env var-files; isolated S3 state per env
- Right-sized per environment; canary on preprod/prod

CI/CD with Harness — gated promotion

CI · build + scan



CD · promote



Promotion gates

- Smoke tests (dev) → ZAP DAST clean + approval (sit)
- Canary verified + approval (preprod) → 2-person approval + canary (prod)
- Auto-rollback on canary failure (25% → 10-min verify → 100%)

Walkthrough — bring it up & ingest



bash — quick start

```
→ docker compose up -d
✓ 10 services healthy (rag, eval, transform, spring, ...)

→ python scripts/seed_data.py --source ./data/sample-docs/
Seeded 42/42 chunks ✓

→ python scripts/ingest_sources.py --list
Available sources: confluence, sharepoint

→ python scripts/ingest_sources.py --source confluence
Ingesting from 'confluence' → http://localhost:8001
Ingested 2 documents → 3 chunks [sample (offline fallback)] ✓

→ python scripts/ingest_sources.py --source sharepoint
Ingested 2 documents → 4 chunks [sample (offline fallback)] ✓
```

What just happened

- Full local stack on Docker
- Sample corpus seeded into RAG
- Confluence + SharePoint connected
- Offline fallback → no creds needed
- Index now holds wiki + sample docs

Set CONFLUENCE_* / SHAREPOINT_*
for live ingestion

Query → grounded, gated answer

```

bash — query

→ curl -X POST localhost:8001/query \
→   -d '{"query": "What is our Q2 revenue forecast?"}'

{
  "answer": "The Q2 APAC revenue forecast
            is AUD 4.2M, based on the
            confirmed pipeline.",
  "sources": [{"title": "Q2 Forecast",
                  "url": "confluence/.../100021"}],
  "sufficiency_score": 0.91,
  "hops": 2,
}

```

Eval gate result

Answer relevancy	≥ 0.80	0.93
Faithfulness	≥ 0.85	0.90
Contextual recall	≥ 0.70	0.81
Hallucination	≤ 0.10	0.03
Toxicity	≤ 0.05	0.00

✓ **PASS — answer released to user**

Quality, testing & security

99

automated tests

unit + integration, all green

0

ruff lint issues

consistent, clean code

100%

mocked I/O

tests need no infrastructure

0

HIGH/CRITICAL CVEs

gated by remediation flow

Testing strategy

- External services stubbed in conftest
- DeepEval suite gated, not run by default
- Connector parsing covered by 24 tests
- Eval gate doubles as a CI quality gate

Security posture

- SAST (Semgrep, Gitleaks, OWASP) in CI
- Trivy image scanning before ECR push
- OWASP ZAP DAST against SIT
- PII redaction in the telemetry pipeline

Roadmap & next steps

- More connectors: Jira, Google Drive, S3, Slack
- Graph enrichment on ingest (entities → Neo4j)
- Incremental / scheduled sync with change detection
- Per-source access control + row-level filtering
- Alerting on the single-pane SLOs (error budget burn)

Delivered this iteration

- Single pane of glass + RED metrics
- Confluence & SharePoint connectors
- /ingest API + CLI, 24 new tests
- Test-collection fix (99 green)

Thank you

Questions & live demo · docs/slides/ai-factory-demo.pdf